Bill Creation Automation with Claude: Research and Content Plan

Business Process Automation · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> Client use case 4, 'Bill Creation Automation': 'Creation of bill for vendor as we received mail from candidate. Check the amount and create with GST and without GST bill as per requirement of company.' The client asked for each use case to be taught as 'Automation of <> using Claude'.

In staffing and IT services firms, a placed candidate's confirmation mail starts a billing chain that is still largely manual: someone reads the mail, finds the agreed rate, retypes the amount, decides whether GST applies and how, numbers the document and mails it. Each step is a chance for a wrong figure, a wrong tax head or a duplicate. The client's request is exactly this: read the mail, check the amount, and create the bill with or without GST as the company requires.

GST documents are not free-form. Rule 46 sets the fields a tax invoice must carry, a bill of supply replaces it for exempt-only or composition suppliers, serial numbers must be consecutive and unique for the financial year [1], and the choice between IGST and CGST plus SGST follows from where the supplier and place of supply are [14]. Larger issuers must also e-invoice [24]. A language model is good at reading mails and attachments [6] and explaining discrepancies, and poor at being the system of record for tax arithmetic. The course is built on that split.

Learners leave with a pipeline in which Claude reads and drafts, tested Python computes and numbers, a named person approves through ServiceNow or a review CLI [21], and an append-only log records everything [26]. The design is deliberately conservative so finance teams can adopt it, and it reuses the same pattern as the other automation courses in the series.

2. How the outline was shaped

Decision	Rationale
Claude never computes tax or amounts; a pure Python engine does.	The client's 'check the amount' and 'with GST and without GST' are arithmetic and rule decisions. Decimal arithmetic with an explicit rounding mode [17] and property-based tests [18] give the certainty finance needs; Claude explains results instead.
'With GST and without GST' is taught as a legal consequence, with both previews drafted and only the applicable one issued.	Whether a bill carries GST depends on the issuer's registration and the supply: tax invoice, bill of supply for exempt-only or composition suppliers [1], or a non-GST bill below the registration threshold [15]. Drafting both previews from one source makes the difference visible to the reviewer without ever issuing the wrong one.
Read-only MCP tools first; send and issue only behind an approval gate and a hook.	The MCP specification calls for a human able to deny tool invocations [10]; CLAUDE.md is not enforced [11], so a PreToolUse hook blocks send and issue tools [12].
Mail intake through Microsoft Graph or IMAP, with Gmail as a reference alternative.	The client said 'as we received mail'; corporate mailboxes are mostly Microsoft 365 [2] [3] or IMAP [20], and Gmail APIs are shown for teams on Google Workspace [4] [5].
Gap-free numbering advanced only on the approved path.	Serials must be consecutive and unique per financial year and are reported in GSTR-1 [1]; allocating numbers at draft time creates gaps.
Approval in ServiceNow through the Service Catalog API, with a CLI fallback, and export to CSV or Tally XML.	The audience runs ServiceNow for requests [21] [22] and many Indian finance teams post in Tally [23]; both are reached through documented interfaces rather than screen automation.

3. Audience and prerequisites

Role	What they need	Starting point
Finance operations and shared-services engineers	Automate billing without losing control of tax treatment and numbering.	Know GST invoicing (GSTIN, CGST, SGST, IGST, HSN and SAC); basic Python.
Enterprise IT and network operations teams	Host the service, mailbox integration, secrets and CI.	Strong on infrastructure and Microsoft 365 or IMAP; new to GST rules and agent tooling.
Automation and platform engineers	Build the MCP server, Agent SDK agent, FastAPI service and ServiceNow and Tally integrations.	Comfortable with Python, JSON and REST APIs.
Finance controllers and tax reviewers (optional attendees)	See how rates, approvals and audit evidence are controlled.	Expert in GST; not developers.

Prerequisites from the outline

- Basic Python and JSON
- Working knowledge of GST invoicing: GSTIN, CGST, SGST, IGST, HSN and SAC codes
- Familiarity with a corporate mailbox (Microsoft 365 or IMAP) and a vendor master
- Familiarity with an AI coding assistant is helpful, not required

Checks before enrolment

- Can you explain when CGST plus SGST applies and when IGST applies?
- Can you read a GSTIN and say which State it belongs to?
- Can you create a Python virtual environment, install packages and run pytest?
- Do you have access to a test Microsoft 365 mailbox or IMAP account, or will you use the course's synthetic mailbox only?
- Can you obtain a Claude API key or Claude Code access for the course days?
- Has your finance team agreed that only fictional vendors and synthetic mails will be used?

Course outcomes

- Describe the vendor bill workflow end to end and separate what Claude drafts from what deterministic code computes
- Set up Claude Code, the Claude Agent SDK and an MCP server that exposes mailbox and vendor-master tools with read-only defaults
- Extract candidate, vendor, amount and GSTIN fields from mail into strictly validated structured JSON outputs
- Reconcile claimed amounts against a vendor rate card and compute CGST, SGST and IGST in tested Python, never in the model
- Draft paired with-GST and without-GST previews, issue only the document that applies, gate issuance behind human approval, and keep a complete audit trail

4. Lab environment

Each learner runs the billing service, MCP server and SQLite database locally or on a lab VM, with Claude Code and the Agent SDK on the host. Mail comes from a synthetic mailbox (a local IMAP server or exported MIME files) with an optional Microsoft 365 developer tenant for Graph. Approvals use a ServiceNow developer instance or the review CLI. Exports target a TallyPrime test company or files. All vendors and candidates are fictional.

Components

Component	Version or edition	Purpose	How to obtain	Notes
Claude Code	Current release at course date	Agentic coding, CLAUDE.md, hooks and MCP client.	Install from the official Claude Code documentation with an organization account or API key.	Hooks and project MCP scope used from module 2 [12] [13].
Claude Agent SDK for Python	claude-agent-sdk, current release	Intake, extraction and explanation agents.	pip install claude-agent-sdk in a virtual environment [9].	API key in environment.
Python with Pydantic, FastAPI, pytest and Hypothesis	Python 3.11 or later; Pydantic 2.x; current FastAPI, pytest, Hypothesis	Pipeline, engine and tests.	pip install pydantic fastapi uvicorn pytest hypothesis.	Decimal for all money [17]; Hypothesis for properties [18].
SQLite	3.47 or later recommended	Vendor master, rate cards, sequence and audit log.	Bundled with Python; sqlite3 CLI from sqlite.org.	3.47 allows expressions in RAISE messages [26].
Headless Chromium with Puppeteer	Current Puppeteer release	HTML to PDF rendering.	npm install puppeteer in the renderer folder.	Pin the version and fonts for stable golden PDFs [8].
Mailbox	Synthetic IMAP server or Microsoft 365 developer tenant	Mail intake and dispatch.	Course image includes a local IMAP server with the synthetic mailbox; optional Microsoft 365 developer tenant for Graph.	Graph permissions Mail.ReadBasic/Mail.Read and Mail.Send [2] [3] [19]; IMAP over TLS on 993 [20].
ServiceNow developer instance	Current release	Approval through a catalog item.	Free personal developer instance from the ServiceNow developer program.	Trainer provides the catalog item definition [21] [22]; CLI fallback available.
TallyPrime	Current TallyPrime release (educational or trial mode)	Testing the XML voucher import.	Trainer machine with a test company, or file-only export for learners.	HTTP server on port 9000 [23].

Genpoor Tech TRAINING INNOVATE BUILD For more info: https://genpoor.com

GitHub and GitHub Actions	github.com	Pull requests and CI.	Course template repository.	Golden tests in module 8.

Fixtures and sample data

Fixture	Contents	Used in
Synthetic mailbox	Twenty confirmation threads with PDF attachments, two missing GSTINs, one with two amounts, one encrypted PDF, one forwarded duplicate.	The vendor bill workflow and the agentic baseline; Mail intake and structured extraction
Fictional vendor master and rate cards	Vendors with GSTINs across Maharashtra and Karnataka, regular, composition and unregistered types, versioned rate cards including NS-2026-03.	Vendor master, amount verification and reconciliation; The GST computation engine
SAC and rate table	Versioned mapping of service types to SAC codes and rates, owned by the finance role.	The GST computation engine; Testing, CI and accounting hand-off
Golden cases and PDFs	Hand-checked figures and reference PDFs for both templates.	The GST computation engine; Bill generation, numbering and templates; Testing, CI and accounting hand-off
Capstone mailbox	Unseen threads with mixed cases.	Capstone: end-to-end bill creation pipeline

Lab safety rules

- Only fictional vendors, GSTINs and synthetic mails are used; no real tax documents.
- Dispatch is limited to the lab mail domain by an allow-list in the MCP server and service.
- No real Tally company or production ServiceNow instance is connected.
- Credentials live in environment variables or a secrets manager, never in prompts, CLAUDE.md, .mcp.json or git.
- Mail and attachments are treated as untrusted input; extraction runs without tools.
- Tax figures produced in the course are teaching examples; rates and SAC mappings must be confirmed by the client's tax advisor before production use [25].

5. Module-by-module content plan

Day 1

Module 1: The vendor bill workflow and the agentic baseline

Goal. Learners map how a placed candidate's confirmation mail becomes a bill today, list what a compliant GST document must carry, and decide when a tax invoice or a bill of supply applies.

Outline topics.

- How a placed candidate's confirmation mail becomes a vendor bill today: intake, amount check, GST decision, numbering, dispatch
- What a compliant tax invoice must carry: GSTIN of supplier and recipient, place of supply, HSN or SAC, rate split, invoice number and date
- When the company issues a tax invoice with GST and when a bill of supply without it: exempt or nil-rated supplies, or when the issuing entity is not registered or under composition
- The teaching pattern: Claude plans, drafts and explains; deterministic code computes and issues; a person approves; everything is logged

Learning objectives.

- Learners can trace the manual flow from confirmation mail to dispatched bill and name every decision and failure point.
- Learners can list the mandatory Rule 46 tax invoice fields and the bill of supply fields.
- Learners can decide, from the issuer's registration status and the supply, whether a tax invoice or a bill of supply applies and record why.
- Learners can explain the course pattern: Claude reads and drafts, code computes and issues, a person approves, everything is logged.

The flow as it runs today

- A placed candidate (or the staffing vendor on their behalf) mails a joining confirmation; finance opens it, finds the rate, retypes the amount, picks a tax treatment and numbers the bill.
- Record each hand-off: intake, amount check against the agreement, GST decision, numbering, PDF, dispatch, posting to accounts.
- Typical failures: wrong rate card line, missing or mistyped GSTIN, CGST plus SGST used on an inter-State supply, duplicate bills from forwarded mails, numbering gaps.
- Clarify with the client who issues the document to whom: the company billing a client for the placement, or a vendor billing the company. The GST logic is the same, but the GSTINs and approval owners differ.
- A services invoice issued after the supply must be issued within 30 days [1]; slow manual intake puts that at risk.

What a compliant tax invoice carries

- GST law prescribes no layout, but Rule 46 fields are mandatory [1].
- Supplier name, address and GSTIN; consecutive serial number unique for the financial year; date; recipient name, address and GSTIN [1].
- HSN or SAC code, description, taxable value, rate and amount of central, State and integrated tax [1].

GenNoor Tech
For more info - https://gennoor.com

- Place of supply with the State name for inter-State supplies, and whether tax is payable on reverse charge [1] [16].
- Signature or digital signature, not required for electronic invoices issued under the IT Act [1].
- Services invoices are prepared in duplicate: ORIGINAL FOR RECIPIENT and DUPLICATE FOR SUPPLIER [1].

Tax invoice or bill of supply

- A registered person making taxable supplies issues a tax invoice [1].
- A registered person dealing only in exempted supplies, or under the composition scheme, issues a bill of supply instead [1].
- A service provider below the registration threshold (Rs 20 lakh, Rs 10 lakh in specified States) need not register [15] and cannot charge GST; the course treats its document as a non-GST bill with the reason recorded.
- No invoice or bill of supply is needed below Rs 200, subject to conditions [1]; the pipeline still records the request.
- Taxpayers whose aggregate turnover exceeded Rs 5 crore must e-invoice from 1 August 2023 [24]; if the client is in scope, an IRP step must be added after approval and before dispatch.

The teaching pattern

- Claude reads mail, extracts fields and writes explanations; it never computes tax or chooses a number.
- Deterministic Python validates, reconciles, computes GST, renders and numbers.
- A named person approves before a number is reserved or a mail is sent.
- Every prompt, tool call, decision and document is logged, as the MCP specification recommends for tool usage [10].

Instructor demo.

1. Open a synthetic confirmation mail thread and walk the manual process aloud.
2. Fill a blank tax invoice field by field and point to the Rule 46 item for each [1].
3. Show the same supply as a bill of supply and highlight what disappears (tax lines) [1].
4. Run the decision table for three issuers: regular registered, composition, unregistered below threshold [1] [15].
5. Mark on the flow where Claude, code and people act.

Hands-on lab. Walk a sample mail thread from the synthetic mailbox through the manual process and record every decision point and failure mode.

1. Open the synthetic mailbox and pick thread BR-0931.
2. List every piece of data the bill needs and where in the thread it appears.
3. Look up the vendor and rate card in the fictional vendor master spreadsheet.
4. Decide the document type and tax treatment and write down why [1] [14].
5. Fill the bill field checklist and mark any field you could not source.

6. Repeat for BR-0932 (missing GSTIN) and BR-0935 (two amounts).
7. Draw the manual flow and mark decision points and failure modes.
8. Mark which steps Claude, code and a person will own in the target flow.

Lab success criteria.

- The field checklist covers every Rule 46 item [1].
- Each thread has a recorded document-type decision with a reason.
- At least five failure modes are identified with where they occur.
- The target flow shows an approval step before numbering and dispatch.

Deliverable. Annotated workflow map and bill field checklist.

Common pitfalls.

- Treating 'without GST' as a choice rather than a consequence of the issuer's registration and the supply's tax status [1].
- Forgetting the place of supply and reverse-charge fields [1].
- Ignoring e-invoicing when the client's turnover puts it in scope [24].
- Mapping the flow without asking who is the supplier on the document.

Knowledge check.

1. When does a registered person issue a bill of supply instead of a tax invoice? Answer: When it deals only in exempted supplies or is under the composition scheme [1].
2. Name four Rule 46 fields that a bill of supply does not need. Answer: Rate of tax, amount of tax, place of supply for inter-State tax purposes and the reverse-charge indication; it keeps supplier, serial number, date and recipient details [1].
3. How many copies of a services invoice are prepared and how are they marked? Answer: Two: ORIGINAL FOR RECIPIENT and DUPLICATE FOR SUPPLIER [1].
4. What changes if the issuer's turnover exceeded Rs 5 crore? Answer: It must e-invoice, so an IRP step is needed before dispatch [24].

Slides and assets to prepare.

- Rule 46 field checklist poster [1].
- Decision table: tax invoice, bill of supply or non-GST bill [1] [15].
- Synthetic mailbox export with twenty threads.
- Fictional vendor master and rate card workbook.
- Slide: who acts at each step (Claude, code, person).

Sources for this module. 1, 10, 14, 15, 16, 24

Module 2: Setting up Claude Code, the Agent SDK and MCP for billing

Goal. Learners scaffold the billing repository, run a Python agent on the Claude Agent SDK, and expose mailbox search, vendor lookup and rate-card fetch through a read-only MCP server.

Outline topics.

- Installing Claude Code and scaffolding the billing repository with CLAUDE.md conventions for this workflow
- A Python agent on the Claude Agent SDK with tool use and a JSON schema for every response
- An MCP server exposing read-only mailbox search, vendor-master lookup and rate-card fetch as tools; no send or write tools yet
- Secrets handling: API keys and mailbox credentials in environment variables, never in prompts or the repository

Learning objectives.

- Learners can scaffold the repository with a CLAUDE.md that states billing conventions and forbidden actions, backed by a hook.
- Learners can build an Agent SDK agent whose every response follows a JSON schema.
- Learners can build an MCP server with read-only tools and output schemas, and no send or write tools.
- Learners can keep API keys and mailbox credentials in the environment and out of prompts and git.

Repository and CLAUDE.md

- Create the repository and run /init, then rewrite CLAUDE.md under 200 lines [11].
- State the rules: money is Decimal, tax is computed only in gst_engine, no invoice number outside the approval path, no mail sending tools.
- Put detail in .claude/rules/ (for example rules/gst.md) or import it with @path [11].
- CLAUDE.md is context, not enforcement; add a PreToolUse hook that denies any tool whose name matches send or issue until approval exists [11] [12].

The Agent SDK agent

- Install claude-agent-sdk in a virtual environment [9].
- Configure ClaudeAgentOptions with allowed_tools set to the read-only MCP tools and disallowed_tools removing Bash and file writes [9].
- Set output_format to a json_schema derived from the Pydantic model with model_json_schema() [9] [27].
- Use query() for batch intake and ClaudeSDKClient for the reviewer's interactive questions [9].
- Re-validate every response with model_validate_json() before use [27].

The read-only billing MCP server

- Tools: search_mail(since, sender), get_mail(message_id), get_vendor(gstin or vendor_id), get_rate_card(vendor_id, date).
- Each tool has an inputSchema and an outputSchema and returns structuredContent [10].
- Return business errors (vendor not found) with isError: true [10].
- Validate inputs, enforce access control and rate limits, and log every call [10].
- Do not trust tool annotations from servers you do not control [10]; there are no write tools in this module.

Connecting and secrets

- Register with claude mcp add --transport stdio billing -- python -m billing_mcp and choose --scope project to share via .mcp.json [13].
- Approve the project server deliberately when Claude Code prompts [13].
- Mailbox client secret, Claude API key and ServiceNow credentials are environment variables read by the server or a secrets manager.
- Never paste credentials into prompts; Claude does not need them because the server holds them.
- Agent SDK tools are named mcp__billing__<tool>, which is what hooks and allowed_tools match [9] [12].

Instructor demo.

1. Scaffold the repository and show CLAUDE.md and the rules folder [11].
2. Start the MCP server and register it; show /mcp [13].
3. Ask Claude Code for the last ten candidate mails; show the tools/call and structuredContent [10].
4. Ask Claude Code to 'send the bill now' and show the hook deny it [12].
5. Run the Agent SDK script with output_format and print the validated object [9].

Hands-on lab. Build and run the MCP server, connect it to Claude Code, and have the agent list the last ten candidate mails without altering anything.

1. Create the virtual environment and install claude-agent-sdk, pydantic, the MCP Python SDK and pytest.
2. Define MailSummary, Vendor and RateCard models with extra='forbid' [27].
3. Implement the four read-only tools with output schemas against the synthetic mailbox and SQLite vendor master [10].
4. Register the server at project scope and confirm it in /mcp [13].
5. Write CLAUDE.md and rules/gst.md [11].
6. Add the PreToolUse hook that denies send or issue tools [12].
7. Write the Agent SDK script that lists the last ten mails with allowed_tools limited to the MCP read tools [9].
8. Confirm the mailbox fixture is byte-identical before and after the run.

Lab success criteria.

- The agent lists ten mails using only MCP tools.
- The final output parses into the MailSummary list schema.
- The hook blocks any attempt to send or issue.
- The mailbox is unchanged and no secret is present in the repository.

Deliverable. Working Claude Code and Agent SDK environment with a read-only billing MCP server.

Common pitfalls.

- Relying on CLAUDE.md to stop sends instead of a hook [11] [12].
- Listing future write tools in allowed_tools, which bypasses the can_use_tool callback [9].
- Using float in models for amounts.
- Committing .mcp.json with inline secrets.

Knowledge check.

1. Why does this module expose no send or write tools? Answer: So the agent cannot change anything while intake and checks are built; writes come only after the approval gate exists.
2. What does the hook do that CLAUDE.md cannot? Answer: It enforces the rule by denying the tool call; CLAUDE.md is only context [11] [12].
3. What must a tool with an outputSchema return? Answer: structuredContent that conforms to the schema [10].
4. Where do mailbox credentials live? Answer: In the MCP server's environment or a secrets manager, never in prompts or the repository.

Slides and assets to prepare.

- CLAUDE.md and rules/gst.md templates [11].
- Hook script deny_send_issue.py [12].
- Architecture diagram: Claude Code, Agent SDK, billing MCP server, mailbox, SQLite.
- Starter MCP server code with four read-only tools [10].
- Secrets checklist.

Sources for this module. 9, 10, 11, 12, 13, 27

Module 3: Mail intake and structured extraction

Goal. Learners read confirmation mails through Microsoft Graph or IMAP, pass PDF and image attachments to Claude, and produce a strictly validated BillRequest with unknowns marked.

Outline topics.

- Reading mail through the Microsoft Graph API or IMAP, including PDF and image attachments passed to Claude for extraction

Genmoor Tech TRAIN · INNOVATE · BUILD For more info - https://genmoor.com

- A typed BillRequest record: candidate, vendor, amount, currency, GSTIN, service period, source message id, confidence per field
- Structured JSON outputs validated by Pydantic; unknown or conflicting fields are marked, never guessed
- Rejected-mail reporting and a queue for human clarification

Learning objectives.

- Learners can read messages and attachments through Microsoft Graph with least privilege, or through IMAP read-only.
- Learners can send PDF attachments to Claude within the documented limits.
- Learners can extract a BillRequest with per-field confidence and explicit unknown and conflict states.
- Learners can produce a rejection report and a clarification queue.

Reading the mailbox

- Graph: GET /users/{id}/messages pages 10 at a time by default, $top up to 1000; follow the full @odata.nextLink [2].
- List returns HTML bodies only; for text, GET the single message with Prefer: outlook.body-content-type="text" [2] [3].
- GET /messages/{id}/$value returns MIME, which Python's email package can parse into parts and attachments [3].
- Start from the least-privileged permission listed (Mail.ReadBasic) and move to Mail.Read only if the fields you need are not returned [2] [3].
- IMAP alternative: IMAP4_SSL on port 993 with ssl.create_default_context() (the default does not verify certificates), select('INBOX', readonly=True), and UIDs rather than message numbers [20].
- Gmail alternative: messages.list returns ids, messages.get returns details, attachments.get returns the attachment body with gmail.readonly [4] [5].

Attachments to Claude

- Send PDFs as document blocks with media_type application/pdf [6].
- Limits: 128 MB request on the first-party API, 600 pages (100 under a 1M-token context), and no encrypted or password-protected PDFs [6].
- Reject encrypted PDFs before the call and route them to clarification.
- Use the Files API and file_id for large attachments reused across steps [6].
- Images (scanned joining letters) go as image blocks; record the source attachment id for each extracted field.

The BillRequest record

- Fields: candidate_name, vendor_name, vendor_gstin, recipient_gstin, amount, currency, service_period_start, service_period_end, joining_date, source_message_id, attachment_ids.

- Each field holds value, status (found, missing, conflicting), confidence and evidence (quoted text and location).
- Amounts are strings in the schema and become Decimal in code; numeric bounds are checked in Pydantic because the schema sent to Claude cannot carry them [7] [27].
- Objects need additionalProperties: false; check stop_reason for refusal or max_tokens before trusting output [7].
- Two amounts in one thread is a conflicting status, never a choice.

Rejections and clarification

- Rules: missing GSTIN, conflicting amount, unknown vendor, encrypted attachment, currency not INR.
- Each rejected mail gets a reason code and a draft clarification mail that a person sends.
- The rejection report lists counts by reason per run.
- Mail content is untrusted input: Claude extracts with no tools, and evidence must match the source text.

Instructor demo.

1. List the last day's mails through Graph and show nextLink paging [2].
2. Fetch BR-0931 as MIME and extract its PDF attachment [3].
3. Send the PDF to Claude and show the BillRequest with evidence [6] [7].
4. Show BR-0932 come back with vendor_gstin status missing.
5. Show BR-0935 come back with amount status conflicting (36,000 and 38,000).
6. Run the same intake over IMAP read-only [20].

Hands-on lab. Run twenty synthetic mails through the extractor, including two with missing GSTINs and one with two amounts, and inspect what is flagged.

1. Implement the Graph reader with paging and the IMAP reader with read-only select and UIDs [2] [20].
2. Parse MIME into body text and attachments [3].
3. Define BillRequest with per-field status, confidence and evidence [27].
4. Implement extraction with structured outputs and PDF document blocks [6] [7].
5. Add the evidence check and the stop_reason check [7].
6. Run all twenty mails and write the rejection report.
7. Inspect the two missing-GSTIN mails and the two-amount mail.
8. Write pytest cases for missing, conflicting and encrypted-attachment outcomes.

Lab success criteria.

- Seventeen requests parse and three go to clarification, matching the fixture's answer key.
- No missing or conflicting field is filled with a guess.
- Every extracted field has evidence pointing to body or attachment text.

- The mailbox is unchanged after the run.

Deliverable. Validated intake pipeline with a rejection report.

Common pitfalls.

- Building $skip by hand instead of following @odata.nextLink [2].
- Leaving IMAP certificate verification off because the default context does not verify [20].
- Selecting the IMAP inbox read-write and marking messages seen.
- Sending encrypted PDFs and treating the error as an extraction failure [6].
- Converting amounts to float during parsing.

Knowledge check.

1. How do you page through Graph messages correctly? Answer: Apply the whole @odata.nextLink URL to the next request; do not extract $skip [2].
2. What must you pass to IMAP4_SSL to verify the server certificate? Answer: An ssl_context from ssl.create_default_context() [20].
3. What is the page limit for a PDF request to Claude? Answer: 600 pages, or 100 when the context window is under 1M tokens [6].
4. What does the extractor return when a thread contains two amounts? Answer: The amount field with status conflicting and both candidates as evidence; the request goes to clarification.

Slides and assets to prepare.

- Graph and IMAP reader reference code [2] [3] [20].
- BillRequest JSON Schema poster.
- Twenty-mail synthetic mailbox with answer key.
- Rejection reason codes and clarification mail templates.
- Slide: PDF limits and document blocks [6].

Sources for this module. 2, 3, 4, 5, 6, 7, 20, 27

Day 2

Module 4: Vendor master, amount verification and reconciliation

Goal. Learners load the vendor master and rate card into SQLite, validate GSTINs, and reconcile claimed amounts against agreed rates with explicit mismatch reasons that Claude explains but never changes.

Outline topics.

- Vendor master and rate card in SQLite: GSTIN, state code, registration status, agreed rates and payment terms
- GSTIN format and check-digit validation, state code derivation and vendor-status lookup in deterministic Python

- Reconciling the claimed amount against the rate card with a configurable tolerance and explicit mismatch reasons
- Claude explains each mismatch in plain language for the reviewer; it does not change the number

Learning objectives.

- Learners can model a vendor master and rate card in SQLite.
- Learners can validate GSTIN format and check character and derive the State code in Python.
- Learners can reconcile claimed amounts against the rate card with a configurable tolerance.
- Learners can have Claude explain a mismatch in plain language without altering numbers.

Vendor master and rate card

- Vendor: vendor_id, legal_name, gstin, state_code, registration_type (regular, composition, unregistered), status, payment_terms.
- Registration is PAN based and State specific, so one vendor PAN can have several GSTINs, one per State [15].
- Rate card: vendor_id, line, role or band, rate, unit (per placement, per month), valid_from, valid_to.
- Store amounts as TEXT holding exact decimals and convert to Decimal in code [17].
- Version the rate card; reconciliation always uses the line valid on the joining or service date.

GSTIN validation

- Structure: State code (2 digits), PAN (10), entity code, and a final check sum character, 15 in total [15].
- Validate with a regular expression, then the check character with the course's reference routine.
- The check algorithm is not given on the official flyer cited here, so test the routine against GSTINs confirmed on the GST portal before trusting it.
- Derive supplier State from the first two digits [15]; compare with the vendor master State.
- A valid format does not mean an active registration; a person confirms status on the GST portal for new vendors.

Reconciliation

- Compute expected amount from the rate card line, quantity and period in Decimal [17].
- Compare with the claimed amount; tolerance is configured per vendor (absolute and percentage).
- Outcomes: OK, MISMATCH (with difference and percentage), NO_RATE_LINE, VENDOR_INACTIVE.
- Every outcome stores the rate card line id used.
- Mismatches go to the reviewer; the claimed amount is never silently replaced.

Claude explains, code decides

- Claude receives the reconciliation record and source mail and writes a two-sentence explanation for the reviewer.
- The explanation must quote the rate card line and the claimed figure exactly; a check compares numbers in the text with the record.
- Claude has no tool that writes amounts.
- If the explanation contradicts the record, it is discarded and the raw record is shown.

Instructor demo.

1. Load the fictional vendor master and show a vendor with two State GSTINs [15].
2. Validate 27AAACN1234F1ZX and show the State code 27 derived [15].
3. Reconcile BR-0931 (OK) and BR-0933 (claimed 61,000, agreed 58,000, +5.2%).
4. Ask Claude for an explanation of BR-0933 and show the number check passing.
5. Tamper the explanation to a wrong figure and show it rejected.

Hands-on lab. Load the fictional vendor master, reconcile the extracted amounts, and trace one over-claim to its rate-card line.

1. Create the SQLite schema for vendors and rate cards.
2. Load the fixture CSVs and verify row counts.
3. Implement GSTIN format, check character and State derivation with unit tests [15].
4. Implement reconcile() with Decimal arithmetic and tolerance [17].
5. Run reconciliation over the seventeen parsed requests.
6. Trace BR-0933 to rate card NS-2026-03 line 4 and record the reason.
7. Generate Claude explanations for mismatches and run the number check.
8. Write pytest cases for OK, MISMATCH, NO_RATE_LINE and VENDOR_INACTIVE.

Lab success criteria.

- Every request has a reconciliation outcome with a rate card line id or a reason.
- The over-claim is traced to the exact line and date.
- Explanations contain only numbers that appear in the record.
- All validation and reconciliation tests pass.

Deliverable. Reconciliation module with mismatch reasons and tests.

Common pitfalls.

- Matching the vendor by name instead of GSTIN.
- Using today's rate card instead of the line valid on the service date.
- Treating a valid GSTIN format as proof of active registration.
- Letting the reviewer's explanation step write back an adjusted amount.

Knowledge check.

1. What do the first two digits of a GSTIN tell you? Answer: The State code of the registration [15].

Gennoor Tech INNOVATE. BUILD. For more info - https://gennoor.com

2. Why can a vendor have several GSTINs? Answer: Registration is PAN based and State specific, one GSTIN per State [15].
3. Which rate card line applies? Answer: The line valid on the joining or service date.
4. What may Claude change in a reconciliation? Answer: Nothing; it only explains the record.

Slides and assets to prepare.

- Vendor master and rate card schema diagram.
- GSTIN anatomy slide [15].
- Fixture CSVs with an over-claim and an inactive vendor.
- Reconciliation outcome codes.
- Explanation number-check code.

Sources for this module. 15, 17

Module 5: The GST computation engine

Goal. Learners implement the GST engine as pure, tested Python that decides intra-State or inter-State, applies the rate from the SAC mapping, handles reverse charge, and produces the no-GST path with a recorded reason.

Outline topics.

- Intra-state versus inter-state supply: CGST plus SGST, or IGST, derived from supplier and place-of-supply state codes
- Rate slabs, HSN and SAC mapping, rounding rules and reverse-charge handling as pure functions
- The without-GST path: bill of supply for exempt or nil-rated supplies, or when the issuing entity is not registered or under composition, with the reason recorded
- Property-based and table-driven pytest coverage across every rate and state pair

Learning objectives.

- Learners can derive intra-State or inter-State supply from supplier State and place of supply.
- Learners can compute CGST and SGST or IGST from a SAC rate table with exact rounding.
- Learners can implement reverse charge and the without-GST path with reasons.
- Learners can prove correctness with table-driven and property-based tests.

Nature of supply

- Intra-State: supplier location and place of supply in the same State or Union territory; inter-State: different States or Union territories [14].
- For B2B services outside the specified categories, the default place of supply is the registered recipient's location [14].
- Supplier State comes from the supplier GSTIN, recipient State from the recipient GSTIN [15].

Gemmor Tech
INNOVATE. BUILD.
For more info : https://gemmor.com

- Specified service categories (immovable property, events, transport and others) have their own rules; the engine refuses them unless a rule is configured.

Rates, SAC and rounding

- Map service type to SAC and rate in a versioned table owned by finance; the code never hard-codes rates.
- Manpower supply has been held to fall under SAC 998519 at 18% in an advance ruling [25]; an advance ruling binds only its applicant, so the client's tax advisor confirms the mapping.
- Intra-State: CGST and SGST each at half the rate; inter-State: IGST at the full rate, broadly equal to CGST plus SGST [14].
- Use Decimal and quantize to two places with the rounding mode finance signs off, for example ROUND_HALF_UP; never rely on the default ROUND_HALF_EVEN by accident [17].
- Decide and test whether rounding is per line or on the total; both previews use the same rule.

Reverse charge and the without-GST path

- For notified reverse-charge categories the recipient pays the tax; the invoice must state that tax is payable on reverse charge [16] [1].
- The engine sets reverse_charge true and records the tax the recipient must account for; finance confirms how the template presents it.
- Without-GST path: exempt-only supplier or composition issues a bill of supply [1]; an unregistered supplier below threshold issues a non-GST bill [15].
- The reason is a coded field (EXEMPT_ONLY, COMPOSITION, UNREGISTERED_BELOW_THRESHOLD) and appears on the document.
- The output is one TaxBreakdown object: taxable value, lines, CGST, SGST, IGST, total, reverse_charge, document_type, reason.

Testing the engine

- Table-driven tests: every rate slab against intra-State and inter-State pairs, plus each without-GST reason.
- Property-based tests with Hypothesis: CGST equals SGST, intra-State IGST is zero, inter-State CGST and SGST are zero, total equals value plus taxes [18].
- Hypothesis generates 100 examples per property by default and reports a minimal failing case [18].
- Golden cases from finance with hand-checked figures, including BR-0931: 84,000 at 18% intra-State gives CGST 7,560 and SGST 7,560.

Instructor demo.

1. Compute BR-0931 (Maharashtra to Maharashtra) and show CGST 7,560, SGST 7,560, IGST 0 [14].
2. Change the recipient GSTIN to Karnataka and show IGST 15,120 [14].
3. Compute an exempt-only supplier case and show a bill of supply with reason [1].

4. Show a ROUND_HALF_EVEN versus ROUND_HALF_UP difference on a crafted amount [17].
5. Run the Hypothesis properties and show a deliberately broken engine fail with a minimal example [18].

Hands-on lab. Implement the engine with Claude Code, review every diff, and make the test suite prove intra-state, inter-state and exempt cases.

1. Write the TaxBreakdown model and the SAC rate table loader.
2. Ask Claude Code to draft nature_of_supply() and review the diff line by line [14].
3. Implement compute_gst() with Decimal and the configured rounding [17].
4. Implement the reverse-charge flag and the without-GST path with reason codes [16] [1].
5. Write table-driven tests for every slab and State pair in the fixture.
6. Write Hypothesis property tests [18].
7. Add finance's golden cases.
8. Run coverage and close any gap in branches.

Lab success criteria.

- Intra-State, inter-State, reverse-charge and each without-GST case pass.
- Property tests pass with the default example count [18].
- No float appears anywhere in the engine.
- Every reviewed diff has a reviewer note in the pull request.

Deliverable. Tested GST engine with a full tax breakdown object.

Common pitfalls.

- Using float and getting off-by-one-paisa totals [17].
- Deriving State from a typed address instead of the GSTIN [15].
- Hard-coding 18% instead of reading the SAC table.
- Treating an advance ruling as binding on everyone [25].
- Applying the default place-of-supply rule to a specified service category [14].

Knowledge check.

1. When is IGST charged instead of CGST plus SGST? Answer: When the supplier location and place of supply are in different States or Union territories [14].
2. What is the default B2B place of supply for ordinary services? Answer: The location of the registered recipient [14].
3. Why is rounding mode set explicitly? Answer: Decimal's default context rounds ROUND_HALF_EVEN, which may differ from the policy finance signs off [17].
4. Name one property-based test of the engine. Answer: For intra-State supplies CGST equals SGST and IGST is zero [18].

Slides and assets to prepare.

Gennoor Tech
LEARN · INNOVATE · BUILD
for more info https://gennoor.com

- Nature-of-supply decision flow [14].
- SAC and rate table fixture with version history.
- TaxBreakdown JSON Schema.
- Golden case workbook from finance.
- Slide: rounding modes compared [17].

Sources for this module. 1, 14, 15, 16, 17, 18, 25

Module 6: Bill generation, numbering and templates

Goal. Learners render tax invoice and bill of supply previews from the same BillRequest, serve them from a FastAPI billing service, and hold a gap-free invoice sequence that only the approved path advances.

Outline topics.

- Two versioned templates, tax invoice and bill of supply, rendered to PDF by deterministic code from the same BillRequest
- A FastAPI billing service with endpoints to draft, preview and (after approval) issue the one bill that applies
- Sequential, gap-free invoice numbering held in SQLite and advanced only on the approved path
- Claude drafts the covering mail and a one-paragraph reviewer summary; the numbers come from the engine

Learning objectives.

- Learners can build two versioned templates that carry every required field.
- Learners can render deterministic PDFs from HTML templates.
- Learners can expose draft, preview and issue endpoints with issue gated on approval.
- Learners can implement gap-free numbering unique for the financial year.

Two templates, one source

- tax-invoice v3 and bill-of-supply v2 are HTML templates with a field list checked against Rule 46 [1].
- Both render from the same validated BillRequest and TaxBreakdown; the bill of supply has no tax lines and shows the reason.
- Services copies are marked ORIGINAL FOR RECIPIENT and DUPLICATE FOR SUPPLIER [1].
- Templates are versioned files in git; the version is printed in the footer and stored with each preview.

Rendering to PDF

- Render HTML with a template engine, then print with headless Chromium via Puppeteer's page.pdf() [8].

- page.pdf() uses the print media type; design print CSS or call emulateMediaType('screen') first [8].
- Pin fonts and the renderer version so the same input gives the same layout.
- Previews are watermarked DRAFT and show 'number reserved on approval'.

The billing service

- POST /bills/draft creates both previews and returns their ids.
- GET /bills/{id}/preview?type=tax_invoice or bill_of_supply returns the PDF.
- POST /bills/{id}/issue requires an approval id, issues only the document that applies, and returns the number.
- Claude drafts the covering mail and a one-paragraph reviewer summary; every number in them comes from the engine.

Gap-free numbering

- The serial is consecutive, unique for the financial year, and uses letters, numerals, '-' and '/' [1].
- Keep the format within sixteen characters, the limit the same guidance gives for credit and debit note serials [1]; for example INV/26-27/00418.
- The sequence lives in SQLite and is advanced in the same transaction that records the approval and issue.
- Numbers are never reused; a wrong bill is corrected with a credit note, not by deleting the number [1].
- Issued serial numbers are later reported in GSTR-1 [1], so gaps cause questions.

Instructor demo.

1. Draft BR-0931 and open both previews side by side.
2. Diff the two previews field by field: GSTIN, CGST, SGST and the reason line.
3. Call issue without approval and show the 403 and audit event.
4. Show the sequence table unchanged after a failed issue.
5. Show Claude's covering mail and the check that its figures match the engine.

Hands-on lab. Generate the with-GST and without-GST previews for the same request and diff them field by field.

1. Write the two HTML templates and their field lists [1].
2. Render with page.pdf() and fixed fonts [8].
3. Implement the draft and preview endpoints in FastAPI.
4. Implement the issue endpoint that requires an approval id.
5. Implement the sequence table and number allocation in one transaction.
6. Generate both previews for BR-0931.
7. Extract text from both PDFs and produce a field-by-field diff.
8. Have Claude draft the covering mail and reviewer summary and run the number check.

Lab success criteria.

- Both previews carry every field their document type requires [1].
- The diff shows only the expected differences.
- No number is allocated by drafting or previewing.
- The covering mail's figures match the TaxBreakdown exactly.

Deliverable. Billing service producing paired draft previews and reviewer summaries.

Common pitfalls.

- Allocating the invoice number at draft time and creating gaps.
- Rendering with system fonts that differ between laptops and CI.
- Letting Claude write totals in the covering mail from its own reading of the PDF.
- Exceeding sixteen characters with long prefixes [1].

Knowledge check.

1. When is an invoice number allocated? Answer: Only on the approved issue path, in the same transaction as the approval record.
2. Which characters may an invoice serial contain? Answer: Letters, numerals, hyphen and slash [1].
3. How is a wrongly issued bill corrected? Answer: With a credit note referencing it; the number is never reused or deleted [1].
4. What CSS media type does page.pdf() use by default? Answer: print [8].

Slides and assets to prepare.

- Template files tax-invoice v3 and bill-of-supply v2.
- Field coverage matrix against Rule 46 [1].
- FastAPI endpoint reference.
- Numbering scheme slide with a sixteen-character example.
- PDF diff tool script.

Sources for this module. 1, 8

Day 3

Module 7: Guardrails, approvals and audit for bill issuance

Goal. Learners assemble the approval package, route it to a person through a ServiceNow catalog request or a review CLI, enforce hard limits, and keep an append-only audit log.

Outline topics.

- Approval package: source mail, extracted fields, reconciliation result, tax breakdown, both previews, side by side

- Human approval through a ServiceNow requested item created with the Service Catalog API, or a review CLI for teams without ServiceNow
- Hard limits: amount thresholds, duplicate-bill detection, no issuance without an approver identity
- Append-only audit log of prompts, tool calls, approvals and issued documents; retention and reviewer access

Learning objectives.

- Learners can build an approval package that lets a reviewer decide without opening other systems.
- Learners can create a ServiceNow requested item through the Service Catalog API and read the decision back.
- Learners can enforce thresholds, duplicate detection and approver identity in code.
- Learners can make the audit log append-only and define retention and reviewer access.

The approval package

- Source mail, extracted fields with evidence, reconciliation result, TaxBreakdown, both previews, Claude's summary.
- The package is one JSON record plus attachments; its hash is what the approver approves.
- If anything in the package changes after approval, the approval is void.
- The reviewer sees which document will be issued and why.

Approval through ServiceNow or a CLI

- Order the 'Bill approval' catalog item with POST /sn_sc/servicecatalog/items/{sys_id}/order_now, sysparm_quantity 1 and the package reference in variables [21].
- The response gives request_number and request_id [21]; find the requested item with GET /api/now/table/sc_req_item and a sysparm_query on the request [22].
- Poll or receive the approval state and the approver's user from the Table API [22]; store the RITM number and approver with the bill.
- The integration user needs only the roles required to order the item and read sc_req_item [22].
- Teams without ServiceNow use review approve BR-0931, which records the OS user and a typed reason.

Hard limits

- Amount threshold: above it, a second approver is required.
- Duplicate detection: same vendor, candidate and period, or same source message id, blocks issue.
- No issue without an approver identity distinct from the person who triggered the draft.
- Unknown or conflicting fields block issue regardless of approval.
- A PreToolUse hook denies issue and send tools to Claude entirely [12].

The audit log

- Log prompts, model request ids, tool calls, extraction results, decisions, approvals and issued documents with hashes [10].
- Make the table append-only with BEFORE UPDATE and BEFORE DELETE triggers that call RAISE(ABORT, ...), which returns SQLITE_CONSTRAINT [26].
- The triggers only raise; they do not modify rows, avoiding the undefined behavior SQLite warns about [26].
- Define retention with finance and give reviewers read-only access to the log and documents.

Instructor demo.

1. Assemble the package for BR-0931 and walk through it as a reviewer.
2. Order the catalog item and show the REQ number returned [21].
3. Query sc_req_item for the RITM and approval state [22].
4. Submit a bill above threshold and show it blocked pending a second approver.
5. Try to UPDATE an audit row and show the SQLITE_CONSTRAINT error [26].
6. Show the full audit trail for BR-0931.

Hands-on lab. Submit a bill above threshold, see it blocked, approve it as a reviewer, and read the full audit trail for that bill.

1. Implement the package builder and its hash.
2. Create the 'Bill approval' catalog item in the ServiceNow developer instance, or enable the review CLI.
3. Implement order_now and the sc_req_item lookup [21] [22].
4. Implement threshold, duplicate and approver-identity checks.
5. Create the audit table and append-only triggers [26].
6. Submit an above-threshold bill and confirm it is blocked.
7. Approve as two reviewers and issue.
8. Query the audit trail for the bill and export it.

Lab success criteria.

- The above-threshold bill cannot issue with one approval.
- A duplicate submission is blocked.
- Audit rows cannot be updated or deleted.
- The trail shows every prompt, tool call, approval and the issued document hash.

Deliverable. Gated issuance workflow with audit log.

Common pitfalls.

- Approving a package and then letting the data change underneath.
- Expecting order_now to return the RITM number directly; it returns the request [21].

Gemnoor Tech
INNOVATE. BUILD.
For more info - https://gemnoor.com

- Using one shared ServiceNow account so approver identity is lost.
- Writing triggers that modify rows during update or delete [26].

Knowledge check.

1. What does order_now return? Answer: The request number and request id; the RITM is looked up separately [21] [22].
2. What voids an approval? Answer: Any change to the approval package after it was approved.
3. How is the audit table made append-only in SQLite? Answer: BEFORE UPDATE and BEFORE DELETE triggers that RAISE(ABORT) [26].
4. Name three hard limits. Answer: Amount threshold needing a second approver, duplicate detection, and a distinct approver identity.

Slides and assets to prepare.

- Approval package JSON Schema.
- ServiceNow catalog item definition and API samples [21] [22].
- Review CLI usage card.
- Audit table SQL with triggers [26].
- Retention and access policy template.

Sources for this module. 10, 12, 21, 22, 26

Module 8: Testing, CI and accounting hand-off

Goal. Learners put the pipeline under pytest and GitHub Actions, export approved bills to CSV or Tally XML, dispatch mail through Graph only after approval, and write the operator runbook.

Outline topics.

- pytest suites for extraction fixtures, reconciliation and the GST engine; golden PDFs for template regression
- GitHub pull requests with Claude Code drafting changes and GitHub Actions running the suite on every pull request and push
- Exporting approved bills as CSV or Tally XML for posting, and dispatching the mail through the Graph API only after approval
- Operator runbook: re-running a mail, voiding a bill, rotating credentials

Learning objectives.

- Learners can maintain fixture, engine and golden-PDF tests.
- Learners can run the suite in GitHub Actions on every pull request and push, with Claude Code drafting changes.
- Learners can export approved bills as CSV or Tally XML and dispatch mail with Graph after approval.
- Learners can write a runbook for reruns, voids and credential rotation.

Gennoor Tech
For more info - https://gennoor.com

Test suites

- Extraction fixtures: twenty mails with expected BillRequests.
- Reconciliation and GST engine: table-driven and Hypothesis tests [18].
- Golden PDFs: compare extracted text and field positions rather than raw bytes, because PDF metadata varies.
- Approval and audit tests: issue without approval fails; audit updates fail [26].

Pull requests and Actions

- Claude Code drafts changes on a branch; a person reviews every diff.
- The workflow runs pytest on pull_request and push and uploads failing PDF diffs as artifacts.
- Rate table and template changes require finance review in the pull request.
- A rate slab change that breaks golden cases must fail the build before merge.

Accounting export and dispatch

- CSV export: one row per issued bill with number, date, GSTINs, taxable value, taxes and total.
- Tally: build an ENVELOPE with TALLYREQUEST Import Data and REPORTNAME Vouchers, VOUCHER elements inside TALLYMESSAGE, and post to the TallyPrime HTTP server on port 9000 or hand the file to finance [23].
- Check the response STATUS (1 success, 0 failure with CODE and DESC) and log it [23].
- Dispatch with POST /users/{billing-mailbox}/sendMail and a fileAttachment, using Mail.Send [19].
- 202 Accepted means accepted, not delivered; record the send and watch for bounces [19].
- With application permissions, ask the Exchange administrator to restrict the app to the billing mailbox.

Operator runbook

- Re-running a mail: idempotent by source message id; a rerun never creates a second bill.
- Voiding a bill: issue a credit note referencing the original serial; never delete [1].
- Rotating credentials: Graph client secret, ServiceNow user, Claude API key, Tally access.
- If the client is in e-invoicing scope, the runbook includes IRP failures and retries [24].

Instructor demo.

1. Open a pull request that changes the 18% slab and show Actions fail the golden tests.
2. Download the PDF diff artifact and explain the failure.
3. Fix the change and merge.
4. Export BR-0931 to Tally XML and post it to a TallyPrime test company [23].
5. Send the approved bill through Graph and show the 202 [19].

Hands-on lab. Open a pull request that changes a rate slab, watch Actions fail the golden tests, fix it and merge.

1. Write the GitHub Actions workflow for pytest on pull_request and push.
2. Add golden PDF tests with text extraction comparison.
3. Ask Claude Code to change a rate slab in the SAC table on a branch.
4. Open the pull request and watch the golden tests fail.
5. Inspect the artifact, correct the change with finance's figure and push.
6. Merge after checks pass.
7. Generate the CSV and Tally XML exports for approved bills [23].
8. Draft the operator runbook sections.

Lab success criteria.

- The broken change cannot merge with failing checks.
- The fixed change passes all suites.
- Exports contain only approved, issued bills.
- The runbook covers rerun, void and credential rotation.

Deliverable. CI pipeline, accounting export and operator runbook.

Common pitfalls.

- Comparing PDF bytes and getting failures from timestamps.
- Exporting drafts to accounts.
- Treating Graph's 202 as proof of delivery [19].
- Granting application Mail.Send to the whole tenant without restricting mailboxes.

Knowledge check.

1. What does a 202 from sendMail mean? Answer: The request was accepted; processing and delivery are not yet confirmed [19].
2. What tells you a Tally import succeeded? Answer: A response with STATUS 1; failures return 0 with CODE and DESC [23].
3. How is a wrong issued bill voided? Answer: With a credit note referencing the original; the number is kept [1].
4. Why compare extracted text in golden PDF tests? Answer: Raw PDF bytes vary with metadata such as creation time.

Slides and assets to prepare.

- GitHub Actions workflow YAML.
- Golden PDF test helper.
- Tally XML sample voucher [23].
- Graph sendMail request sample [19].
- Operator runbook template.

Sources for this module. 1, 18, 19, 23, 24, 26

GennoorTech
for more info - https://gennoor.com

Module 9: Capstone: end-to-end bill creation pipeline

Goal. Teams wire intake, reconciliation, GST engine, templates and the approval gate into one runnable pipeline, issue the one bill that applies only after approval, and walk the audit trail.

Outline topics.

- Wire intake, reconciliation, GST engine, templates and approval gate into one runnable pipeline on the synthetic mailbox
- Mail in, both previews drafted, the one bill that applies issued, with no issuance without approval
- Audit trail walk-through: every prompt, tool call and approval for one issued bill
- Acceptance criteria review, limitations statement, team demo

Learning objectives.

- Learners can run the full pipeline on unseen mails.
- Learners can demonstrate that no bill is numbered or sent without approval.
- Learners can walk the audit trail for one issued bill and state the pipeline's limitations.

Wiring the pipeline

- Intake, extraction, reconciliation, GST engine, previews, approval, issue, export and dispatch run as one command over the synthetic mailbox.
- Each stage writes to the audit log and can be rerun idempotently.
- Clarification and mismatch queues are worked by a person before approval.
- The without-GST preview is kept but never numbered when the tax invoice applies.

Acceptance criteria

- Every issued bill has an approval record with approver identity and package hash.
- Tax figures match finance's hand calculation for a sample [14].
- Documents carry every required field for their type [1].
- No duplicate bills and no numbering gaps.

Audit walk-through and limitations

- Pick one bill and show every prompt, tool call, approval and the issued hash [26].
- State the limits: SAC mapping owned by finance, GSTIN status checked by a person, specified service categories not handled, e-invoicing not included unless added [24].
- State who owns each part after handover.
- Demo to a reviewer panel acting as finance.

Instructor demo.

1. Show the capstone brief and acceptance checklist.
2. Run a short end-to-end pass on two mails as a model.

Gennoor Tech TRAINING INNOVATE BUILD For more info: https://gennoor.com

3. Show an audit walk-through the panel will expect.
4. Show a good limitations statement.

Hands-on lab. Capstone build and demo.

1. Receive the unseen capstone mailbox.
2. Run intake and work the clarification queue.
3. Run reconciliation and resolve mismatches with the reviewer.
4. Compute GST and generate paired previews.
5. Route packages for approval and issue approved bills.
6. Export to CSV or Tally XML and dispatch approved mails.
7. Walk the audit trail for one bill.
8. Present results, limitations and runbook.

Lab success criteria.

- Zero bills numbered or sent without approval.
- Sample tax figures match the hand calculation.
- Every issued document carries its required fields.
- The audit walk-through is complete and the panel accepts the limitations statement.

Deliverable. End-to-end pipeline and runbook.

Common pitfalls.

- Skipping the clarification queue to finish faster.
- Approving your own team's bills.
- Leaving the without-GST preview numbered by mistake.

Knowledge check.

1. What is the capstone's hard pass condition? Answer: No bill numbered or sent without an approval record.
2. What happens to the preview that does not apply? Answer: It is retained for the record but never numbered or sent.
3. Name two stated limitations of the pipeline. Answer: SAC mapping and GSTIN status depend on finance and a person; e-invoicing is not included unless added [24].

Slides and assets to prepare.

- Capstone brief and unseen mailbox.
- Acceptance checklist.
- Audit walk-through script.
- Demo schedule.

Sources for this module. 1, 14, 24, 26

Genmoor Tech TRAIN INNOVATE BUILD For more info - https://genmoor.com

6. Capstone plan

A repeatable pipeline that reads a placed candidate's confirmation mail, extracts and validates the bill request, reconciles the amount against the vendor master, computes GST in tested code, drafts both a with-GST and a without-GST preview, and issues only the one document that applies, only after a person approves, with every step logged.

Each team runs the full pipeline on an unseen synthetic mailbox: read the confirmation mails, extract and validate the bill requests, reconcile amounts against the vendor master, compute GST in tested code, draft both the with-GST and without-GST previews, and issue only the document that applies after a person approves, with every step in the audit log.

Milestones

1. Intake run with rejection report and clarification queue worked.
2. Reconciliation complete with every mismatch reviewed.
3. GST computed and paired previews generated.
4. Approvals recorded and applicable bills issued and numbered.
5. Exports produced and approved mails dispatched.
6. Audit walk-through, limitations statement and runbook presented.

Evaluation notes

- Hard fail if any bill is numbered or sent without an approval record.
- The panel recomputes tax for a sample of bills by hand [14].
- Each issued document is checked against the field list for its type [1].
- The sequence is checked for gaps and duplicates.
- The limitations statement must mention e-invoicing applicability and SAC ownership [24] [25].

Assessment weights

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

Gennoor Tech
For more info: https://gennoor.com

7. Promotional video: research behind the script

Opening scenario. The opening (a confirmation mail at ten, a bill needed today, a figure retyped from a spreadsheet and a tax rate picked from memory) is a teaching scenario built from the client's request to read the candidate's mail, check the amount and create a bill with or without GST. The time pressure is realistic because a services invoice issued after supply must be issued within 30 days [1].

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

Claim or on-screen text	Basis
A candidate's mail. The right bill by noon.	Teaching scenario from the client's use case; not a measured turnaround.
9 build steps; 3 days, hands-on	Matches the outline: nine modules across three days.
What a compliant invoice carries	Rule 46 fields as summarised in the official GST flyer [1].
When the company issues with GST and when it issues a bill of supply without it	Bill of supply for exempt-only or composition suppliers [1]; unregistered suppliers below threshold cannot charge GST [15].
An MCP server that exposes your mailbox and vendor master as read-only tools	MCP tools with schemas [10]; read-only design is a course decision enforced with hooks [12].
The agent reads each mail through the Microsoft Graph API or IMAP, including the PDF attachments	Graph list and get message [2] [3]; Python imaplib [20]; Claude PDF support [6].
billagent ingest: 20 mails read, 17 bill requests parsed, 3 sent to clarification	Illustrative output matching the synthetic mailbox fixture's answer key.
BR-0931 Nimbus Staffing INR 84,000.00 gstin 27AAACN1234F1ZX; recipient 27AABCE5678G1ZF	Fictional values. Both have the documented structure: State code 27, a ten-character PAN, entity code and check character [15]. They pass the mod-36 check routine used in the course's reference validator; that algorithm is not on the official flyer cited, so the routine should be confirmed against GSTINs verified on the GST portal.
supplier state 27 (MH), place of supply 27 (MH) → intra-state	Same State for supplier and place of supply means intra-State [14]; default B2B place of supply is the registered recipient's location [14]. Code 27 is Maharashtra in the GST State code list (not stated on the flyers cited).
SAC 998519 rate 18% CGST 7,560.00 SGST 7,560.00 IGST 0.00; total with GST 99,120.00	Arithmetic is correct (9% of 84,000 is 7,560 each). An advance ruling placed manpower supply under SAC 998519 at 18% [25], but advance rulings bind only the applicant and secondary sources disagree on the exact SAC for staffing, so the course treats the mapping as finance-owned configuration.
reconcile BR-0933: claimed 61,000.00 agreed 58,000.00 +5.2% MISMATCH	Illustrative fixture; 3,000 over 58,000 is 5.17%, shown rounded to 5.2%.
Claude explains the computation to the reviewer. It never performs it.	Course design decision; the engine uses Decimal with explicit rounding [17].
Hook narration: someone picks a tax rate from memory	Teaching scenario; the course replaces it with a versioned SAC and rate table.

Preview diff: reason 'GST not charged: exempt supply (no tax lines)'	Illustrative contrast between the two previews. For BR-0931 the supply is taxable, so only the tax invoice may be issued; the bill of supply preview is retained and never numbered, as the cards panel shows [1].
invoice number (reserved on approval only); issued INV/26-27/00418	Serials must be consecutive and unique per financial year using letters, numerals, '-' and '/' [1]; the example has 14 characters, within the sixteen-character limit the same guidance gives for credit and debit notes [1].
A person approves in ServiceNow or in the review CLI; SNOW RITM0048211	Service Catalog order_now returns the request; the RITM is read through the Table API [21] [22]. The RITM number is illustrative.
ci: gst-engine 146/146 passed, templates golden 12/12, PR #57 merged	Illustrative test counts; the mechanism is pytest with Hypothesis properties [18] in GitHub Actions.
Pytest covers the GST engine across every rate slab and every state pair	Achievable for the slabs and State pairs in the fixture tables; 'every State pair' means every pair in the configured table, not an exhaustive legal test.
export: tally_2026-10-28.xml; approved bills export as CSV or Tally XML	TallyPrime XML voucher import format [23].
mail sent via Graph API	sendMail with Mail.Send; 202 Accepted means accepted, not delivered [19].
audit entries: 23; 0 bills issued without approval	Illustrative count; zero unapproved issues is the capstone's hard pass condition, a design property rather than a statistic. The append-only log uses SQLite triggers [26].
You will feed the agent a mail with a missing GSTIN and watch it refuse to proceed.	Lab outcome of the intake module: the request goes to the clarification queue rather than being guessed.

References

[1] GST Tax Invoice and other such instruments in GST (GST flyer). GST Council / CBIC Directorate General of Taxpayer Services (prepared by NACIN). https://www.gstcouncil.gov.in/sites/default/files/e-version-gst-flyers/Tax_Invoice_and_other_new.pdf
Supports: Workflow, numbering and template modules: tax invoice and bill of supply contents. No invoice format is prescribed but Rule 46 of the CGST Rules makes fields mandatory, including supplier name, address and GSTIN; a consecutive serial number of letters, numerals, '-' and '/' unique for a financial year; date; recipient name, address and GSTIN; HSN or accounting (SAC) code; description; taxable value; rate and amount of central, State and integrated tax; place of supply with State name for inter-State supplies; whether tax is payable on reverse charge; and signature (not required for electronic invoices under the IT Act). A registered person dealing only in exempted supplies or under composition issues a bill of supply instead; no invoice is needed under Rs 200 subject to conditions; a services invoice issued after supply must be within 30 days; services invoices are

prepared in duplicate (ORIGINAL FOR RECIPIENT, DUPLICATE FOR SUPPLIER); credit and debit note serials must not exceed sixteen characters; invoice serial numbers are furnished in GSTR-1.

[2] List messages - Microsoft Graph v1.0. Microsoft Learn. https://learn.microsoft.com/en-us/graph/api/user-list-messages?view=graph-rest-1.0
Supports: Mail intake module: listing messages in the billing mailbox. GET /me/messages or /users/{id}/messages returns 10 messages per page by default and $top accepts 1 to 1000; follow the whole @odata.nextLink URL and do not extract $skip from it; this operation returns bodies only in HTML; Mail.ReadBasic is least privileged (Mail.ReadBasic.All for application permissions), with Mail.Read and Mail.ReadWrite higher privileged.

[3] Get message - Microsoft Graph v1.0. Microsoft Learn. https://learn.microsoft.com/en-us/graph/api/message-get?view=graph-rest-1.0
Supports: Mail intake module: reading one message as text or MIME. GET /me/messages/{id} or /users/{id}/messages/{id} returns the message; appending /$value returns the MIME content instead of a message resource; Prefer: outlook.body-content-type="text" returns body and uniqueBody as text; permissions from least to most privileged are Mail.ReadBasic then Mail.Read (Mail.ReadBasic.All, Mail.Read for application).

[4] List Gmail messages. Google for Developers. https://developers.google.com/workspace/gmail/api/guides/list-messages
Supports: Mail intake module: Gmail alternative for teams on Google Workspace. messages.list returns message objects containing id and threadId; full details need messages.get; the q parameter takes Gmail search syntax such as from:someuser@example.com is:unread.

[5] Method: users.messages.attachments.get | Gmail. Google for Developers. https://developers.google.com/workspace/gmail/api/reference/rest/v1/users.messages.attachments/get
Supports: Mail intake module: downloading PDF attachments from Gmail. GET https://gmail.googleapis.com/gmail/v1/users/{userId}/messages/{messageId}/attachments/{id} with an empty body returns a MessagePartBody; gmail.readonly is one of the accepted scopes.

[6] PDF support. Anthropic (Claude Developer Platform docs). https://platform.claude.com/docs/en/build-with-claude/pdf-support
Supports: Mail intake module: passing PDF attachments to Claude. Claude reads text, tables and charts in standard PDFs; the maximum request size is 128 MB on the first-party Claude API (varies by platform), up to 600 pages per request (100 when the context window is under 1M tokens); PDFs must not be password-protected or encrypted; large PDFs can be uploaded with the Files API and referenced by file_id; PDFs are sent as document blocks with media_type application/pdf.

[7] Structured outputs. Anthropic (Claude Developer Platform docs). https://platform.claude.com/docs/en/build-with-claude/structured-outputs
Supports: Setup and intake modules: schema-constrained BillRequest JSON. The schema goes in output_config.format with type json_schema; objects must set additionalProperties to false; recursive schemas, numeric constraints and string length constraints are unsupported; a refusal or max_tokens stop can yield output that does not match the schema; the deprecated output_format parameter moved to output_config.format; client.messages.parse() accepts a Pydantic model and returns parsed_output.

[8] Page.pdf() method | Puppeteer. Puppeteer (pptr.dev). https://pptr.dev/api/puppeteer.page.pdf
Supports: Templates module: rendering HTML bill templates to PDF. page.pdf() generates a PDF using the print CSS media type and adjusts colours for printing by default; call page.emulateMediaType('screen') first to use screen styling.

[9] Agent SDK reference - Python. Anthropic (Claude Code docs). https://code.claude.com/docs/en/agent-sdk/python
Supports: Setup module: the billing agent on the Claude Agent SDK. Install with pip install claude-agent-sdk in a virtual environment; query() runs a new session per call while ClaudeSDKClient reuses a session; ClaudeAgentOptions includes allowed_tools, disallowed_tools, permission_mode, mcp_servers, output_format (json_schema), can_use_tool, hooks, setting_sources and max_turns; @tool and create_sdk_mcp_server() build in-process MCP tools named mcp__<server>__<tool>; can_use_tool is not invoked for calls auto-approved by allowed_tools.

[10] Tools - Model Context Protocol specification (2025-06-18). Model Context Protocol. https://modelcontextprotocol.io/specification/2025-06-18/server/tools
Supports: Setup module: the read-only billing MCP server. Tools are listed with tools/list and invoked with tools/call; a tool has name, title, description, inputSchema, optional outputSchema and annotations, and clients must treat annotations as untrusted unless the server is trusted; with an outputSchema servers must return conforming structuredContent; execution errors use isError: true; a human SHOULD be able to deny invocations; servers MUST validate inputs and enforce access control and rate limits; clients SHOULD log tool usage for audit.

[11] How Claude remembers your project (CLAUDE.md). Anthropic (Claude Code docs). https://code.claude.com/docs/en/memory
Supports: Setup module: billing conventions in CLAUDE.md. Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md; /init drafts one; @path imports other files; target under 200 lines; .claude/rules/ holds modular rules; CLAUDE.md is context, not enforced configuration, so use a PreToolUse hook to block an action regardless.

[12] Hooks reference. Anthropic (Claude Code docs). https://code.claude.com/docs/en/hooks
Supports: Setup and guardrails modules: enforcing no-send and no-issue rules. Hooks are configured under hooks in settings files; a PreToolUse hook blocks a call with exit code 2 or with hookSpecificOutput permissionDecision deny and a reason; MCP tools are matched as mcp__<server>__<tool>, and mcp__memory__.* matches all tools on a server.

[13] Connect Claude Code to tools via MCP. Anthropic (Claude Code docs). https://code.claude.com/docs/en/mcp
Supports: Setup module: registering the billing MCP server. claude mcp add [options] <name> -- <command> [args...] adds a stdio server; --env passes variables; --scope local, project (.mcp.json at the project root) or user; project servers need approval in interactive sessions; /mcp shows status; verify you trust each server because servers that fetch external content can expose you to prompt injection.

[14] Integrated Goods and Services Tax Act (GST flyer, Chapter 21). GST Council / CBIC (NACIN GST flyers compilation). https://www.gstcouncil.gov.in/sites/default/files/e-version-gst-flyers/51_GST_Flyer_Chapter21.pdf
Supports: GST engine module: intra-State versus inter-State supply and place of supply. A supply is inter-State where the location of the supplier and the place of supply are in two different States or Union territories, and intra-State where they are in the same State or Union territory; the kind of tax (IGST or CGST plus SGST) depends on that; the IGST rate broadly equals the CGST rate plus the SGST rate; for services other than the specified categories the default B2B place of supply is the location of the registered recipient.

[15] Registration under GST Law (GST flyer). GST Council / CBIC Directorate General of Taxpayer Services. https://gstcouncil.gov.in/sites/default/files/e-version-gst-flyers/Registration_under_GST_Law_new.pdf
Supports: Vendor master module: GSTIN structure and registration thresholds. Registration is PAN based and State specific, with one GSTIN per State for a PAN; the 15-digit GSTIN has the State code in the first 2 digits, the PAN in the next 10, two digits for entity code and a final check sum digit; suppliers of services with aggregate turnover below Rs 20 lakh (Rs 10 lakh in specified States) need not register.

[16] Reverse Charge Mechanism (GST flyer). GST Council / CBIC Directorate General of Taxpayer Services. https://gstcouncil.gov.in/sites/default/files/e-version-gst-flyers/Reverse%20charge%20Mechanism.pdf
Supports: GST engine module: reverse-charge handling. Under reverse charge the recipient, not the supplier, is liable to pay tax for notified categories under section 9(3) of the CGST Act (section 5(3) IGST Act); every tax invoice must mention whether tax is payable on reverse charge (section 31 read with Rule 46); a supplier cannot take ITC on inputs used for supplies on which the recipient pays tax.

[17] decimal - Decimal fixed-point and floating-point arithmetic. Python Software Foundation. https://docs.python.org/3/library/decimal.html
Supports: GST engine module: exact money arithmetic and rounding. Decimal represents decimal fractions exactly (0.1 + 0.1 + 0.1 - 0.3 is exactly zero) and is preferred for accounting; quantize() rounds to a fixed exponent with an explicit rounding mode such as ROUND_HALF_UP; the default context uses ROUND_HALF_EVEN.

[18] Hypothesis quickstart. Hypothesis. https://hypothesis.readthedocs.io/en/latest/quickstart.html
Supports: GST engine and testing modules: property-based tests. @given with strategies such as st.integers()

generates inputs (100 by default, configurable with max_examples) and reports a minimal failing example; install with pip install hypothesis.

[19] user: sendMail - Microsoft Graph v1.0. Microsoft Learn. https://learn.microsoft.com/en-us/graph/api/user-sendmail?view=graph-rest-1.0
Supports: Testing, CI and hand-off module: dispatching the approved bill. POST /me/sendMail or /users/{id}/sendMail sends a JSON or MIME message, can include a fileAttachment, saves to Sent Items unless saveToSentItems is false, needs Mail.Send (delegated or application), and returns 202 Accepted, which does not mean processing or delivery has completed.

[20] imaplib - IMAP4 protocol client. Python Software Foundation. https://docs.python.org/3/library/imaplib.html
Supports: Mail intake module: IMAP alternative to Microsoft Graph. IMAP4_SSL uses port 993 by default; with the default ssl_context the server certificate and hostname are not verified, so pass ssl.create_default_context(); select(mailbox, readonly=True) prevents modifications; message numbers change after EXPUNGE so UIDs via the uid command are advised; FETCH responses can contain unsolicited data.

[21] Service Catalog API | ServiceNow. ServiceNow (API reference). https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/c_ServiceCatalogAPI.html
Supports: Guardrails module: requesting approval through a catalog item. POST /sn_sc/servicecatalog/items/{sys_id}/order_now orders a catalog item; sysparm_quantity is required, sysparm_requested_for is optional and variables carries the mandatory item variables; the response returns number, request_number, request_id and sys_id of the order request.

[22] Table API | ServiceNow. ServiceNow (API reference). https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html
Supports: Guardrails module: reading the requested item and approval state. The Table API at /api/now/table/{tableName} supports GET, POST, PUT, PATCH and DELETE; the calling user needs roles for the table; sysparm_query filters with encoded queries, sysparm_fields limits fields, sysparm_limit defaults to 10000; sys_id identifies each record.

[23] XML Integration | TallyPrime Developer. Tally Solutions (TallyPrime help). https://help.tallysolutions.com/xml-integration/
Supports: Testing, CI and hand-off module: Tally XML export. Enable the HTTP Server under F1 (Help) > Settings > Advanced Configuration (default port 9000) and post XML to it; an import uses an ENVELOPE with HEADER TALLYREQUEST Import Data and BODY IMPORTDATA with REQUESTDESC REPORTNAME Vouchers and REQUESTDATA TALLYMESSAGE holding VOUCHER elements; responses carry STATUS 1 for success or 0 with STATUS.LIST CODE and DESC on failure.

[24] RSM India Newsflash: Turnover exemption limit for e-invoicing reduced to INR 5 crores w.e.f. 1 August 2023. RSM India (professional advisory newsflash, 12 May 2023). https://www.rsm.global/india/sites/default/files/media/documents/5.%20RSM%20India%20Newsflash%20-%20Turnover%20exemption%20limit%20for%20e-invoicing%20reduced%20to%205%20crores%20w.e.f%201%20August%202023.pdf
Supports: Workflow and limitations: e-invoicing applicability. CBIC Notification No. 10/2023 - Central Tax dated 10 May 2023 reduced the e-invoicing turnover threshold from INR 10 crores to INR 5 crores with effect from 1 August 2023, after earlier reductions from INR 500 crores (2020) through 100, 50, 20 and 10 crores. Secondary source; the notification itself is on the CBIC site.

[25] Manpower and labour supply services attract 18% GST: Telangana AAR. LiveLaw (report of TSAAR Order No. 38/2022 dated 14.07.2022). https://www.livelaw.in/amp/news-updates/telangana-authority-of-advance-ruling-aar-gst-itc-manpower-and-labour-supply-services-206279
Supports: GST engine module and promo check: SAC and rate for manpower supply. The Telangana Authority for Advance Ruling (M/s Sri Bhavani Developers, TSAAR Order No. 38/2022, 14 July 2022) held that manpower or labour supply services provided by a manpower supply agency fall under SAC 998519 and are taxable at 18%. An advance ruling binds only the applicant; secondary report.

[26] CREATE TRIGGER - SQLite. SQLite. https://www.sqlite.org/lang_createtrigger.html
Supports: Guardrails module: append-only audit log and invoice sequence in SQLite. RAISE(ABORT, msg) inside a trigger terminates the current statement and returns SQLITE_CONSTRAINT with the message; from 3.47.0 the message can be an expression; BEFORE UPDATE or BEFORE DELETE triggers that modify or delete the affected row give undefined results.

[27] Models - Pydantic Validation. Pydantic. https://pydantic.dev/docs/validation/latest/concepts/models/
Supports: Setup and intake modules: BillRequest validation. model_validate() and model_validate_json() validate dicts and JSON; all failures are raised in one ValidationError; extra='forbid' rejects unknown fields; model_json_schema() returns the JSON Schema.

For more info - https://gennoor.com

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com